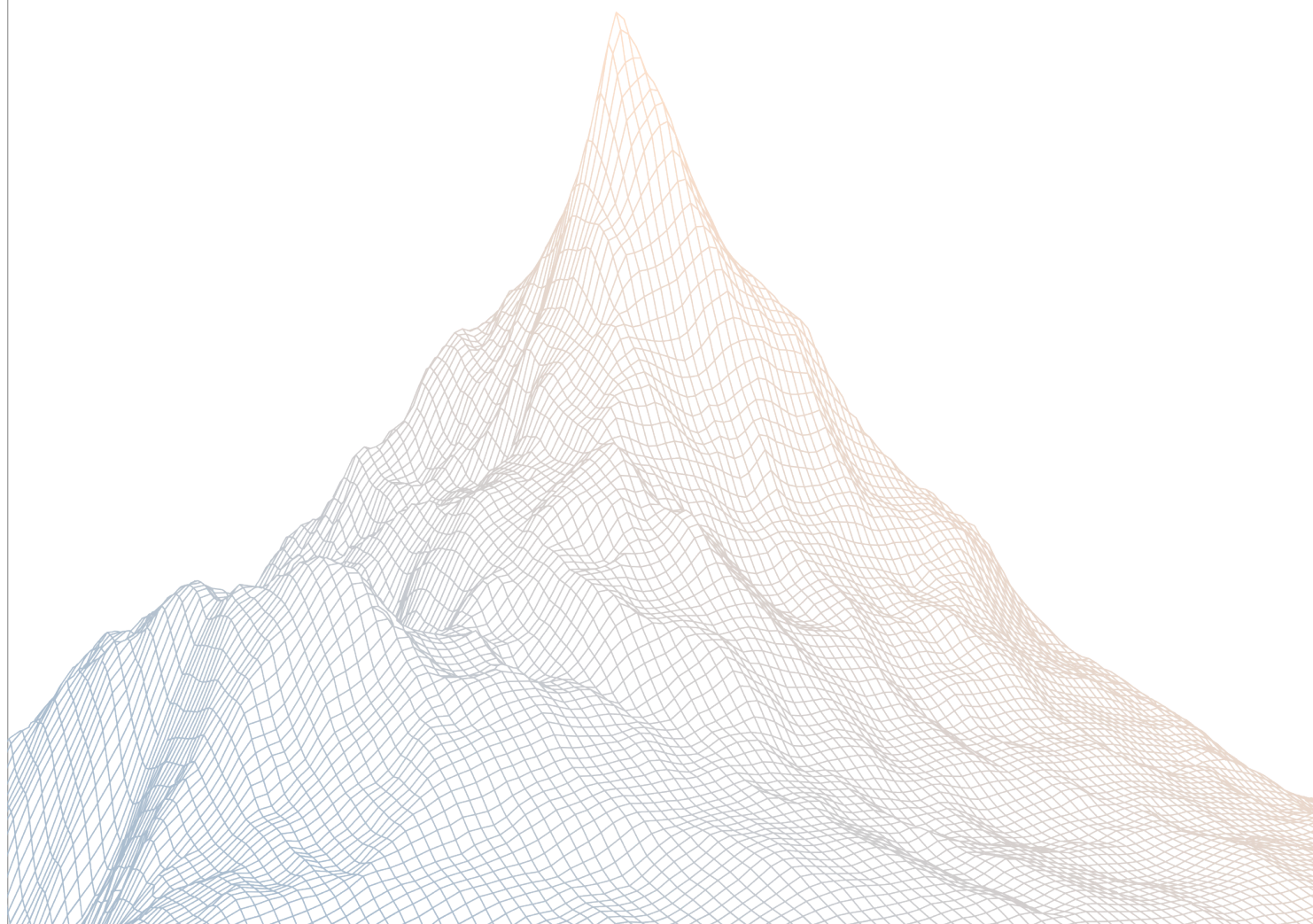


Meteora

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES:

May 6th to May 11th, 2026

AUDITED BY:

J4X
oakcobalt

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Meteora	4
2.2	Scope	4
2.3	Audit Timeline	5
2.4	Issues Found	5
3	Findings Summary	5
4	Findings	6
4.1	Medium Risk	7
4.2	Low Risk	14
4.3	Informational	22

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Meteora

Our mission is to build the most secure, sustainable and composable liquidity layer for all of Solana and DeFi.

By using Meteora's DLMM and Dynamic AMM Pools, liquidity providers can earn the best fees and yield on their capital.

This would help transform Solana into the ultimate trading hub for mainstream users in crypto by driving sustainable, long-term liquidity to the platform. Join us at Meteora to shape Solana's future as the go-to destination for all crypto participants.

2.2 Scope

The engagement involved a review of the following targets:

Target	referral-staking
---------------	------------------

Repository	https://github.com/MeteoraAg/referral-staking.git
-------------------	---

Commit Hash	29bfc910ac23174edc5672e9b97f7f81aab6d097
--------------------	--

Files	programs/referral/src/**/*.*rs
--------------	--------------------------------

Target	Mitigation Review
---------------	-------------------

Repository	https://github.com/MeteoraAg/referral-staking.git
-------------------	---

Commit Hash	aef3a35324eae5f29d2f0bdd91ff4f35bc028190
--------------------	--

Files	programs/referral/src/**/*.*rs
--------------	--------------------------------

2.3 Audit Timeline

May 6, 2026	Audit start
May 11, 2026	Audit end
May 12, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	0
Medium Risk	2
Low Risk	3
Informational	10
Total Issues	15

3

Findings Summary

ID	Description	Status
M-1	Escrow Closure can be DOSd	Acknowledged
M-2	Minimum staking can be bypassed	Acknowledged
L-1	Fee can be distributed across thousands of token accounts	Resolved
L-2	Users can be locked in after staking	Acknowledged
L-3	Protocol-sponsored escrow rent can be captured by owner on close	Acknowledged
I-1	Division and remainder return MathOverflow on division-by-zero	Acknowledged
I-2	Unused Error ReferralError::UnsupportedMintExtension	Resolved
I-3	Unstakes can be DOSd	Resolved
I-4	Missing minimum cooldown bound: intent-enforcement mismatch and protocol policy inconsistency	Resolved
I-5	Flash loan enables event inflation in frictionless staking pools	Acknowledged
I-6	Missing Pubkey::default() guard on fee receiver and owner configuration paths	Resolved
I-7	Manual AccountLoader::close path misses close self-target check	Resolved
I-8	Single-step ownership transfer without new-owner confirmation	Acknowledged
I-9	Slippage on immediate unstake can malfunction	Resolved
I-10	Incorrect receiver account emission	Resolved

4

Findings

4.1 Medium Risk

A total of 2 medium risk findings were identified.

[M-1] Escrow Closure can be DOSd

SEVERITY: Medium

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: High

Target

- [programs/referral/src/instructions/permissionless/ix_close_escrow.rs](#)

Description:

Escrow closure can be permanently grieved because anyone is allowed to stake into any escrow, and `close_escrow` refuses to close an escrow while either `staked_amount` or `partial_unstaked_amount` is non-zero.

The close instruction only closes the escrow if `escrow_state.is_empty()` returns true:

```
pub fn handle_close_escrow(
    ctx: Context<CloseEscrowCtx>,
    skip_close_if_non_empty: bool,
) → Result<()> {
    let escrow_state = ctx.accounts.escrow.load()?;
    let mut staking_state = ctx.accounts.staking.load_mut()?;

    if !escrow_state.is_empty() {
        require!(
            skip_close_if_non_empty,
            ReferralError::CannotCloseNonEmptyEscrow
        );
        return Ok(());
    }

    staking_state.metrics.decrement_total_escrow();
    drop(escrow_state);
}
```

```
ctx.accounts
    .escrow
    .close(ctx.accounts.rent_receiver.to_account_info());
```

The emptiness check depends only on the escrow accounting fields:

```
pub fn is_empty(&self) → bool {
    self.staked_amount == 0 && self.partial_unstaked_amount == 0
}
```

However, staking into an escrow is intentionally permissionless. The StakeCtx does not require the payer to be the escrow owner:

```
#[account(
    mut,
    has_one = staking,
    // Note: We allow anyone to stake into the escrow, so we don't check the
    // owner here
)]
pub escrow: AccountLoader<'info, Escrow>,
```

As a result, an attacker can stake the minimum token amount, such as 1 base unit, into a victim escrow. This increments `staked_amount`:

```
pub fn stake(&mut self, amount: u64) → Result<()> {
    self.staked_amount = self.staked_amount.safe_add(amount)?;
    Ok(())
}
```

The victim must then perform additional unstake/withdraw cleanup before the escrow can be closed again. Since the attacker can repeatedly dust-stake at very low cost, the escrow owner can be repeatedly prevented from closing the account and recovering the rent. This leaves the user unable to recover the escrowed rent.

Recommendations:

We recommend adding a `withdraw_all` parameter to `close_escrow` that allows the owner to atomically sweep any remaining escrow balance and close the escrow in the same instruction.

When `withdraw_all` is true, the instruction should transfer the remaining escrow funds out to the owner or an explicit token receiver, zero the escrow accounting, update staking

totals consistently, and then close the escrow. This prevents an attacker from keeping the escrow non-empty with dust immediately before closure.

The close flow should become conceptually:

```
if !escrow_state.is_empty() {  
    require!(withdraw_all, ReferralError::CannotCloseNonEmptyEscrow);  
  
    // Transfer all remaining staked / partially unstaked funds to the owner.  
    // Update staking and escrow accounting so the escrow becomes empty.  
}  
  
staking_state.metrics.decrement_total_escrow();  
ctx.accounts.escrow.close(ctx.accounts.rent_receiver.to_account_info());
```

Meteora: Acknowledged.

[M-2] Minimum staking can be bypassed

SEVERITY: Medium

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: High

Target

- [programs/referral/src/states/staking.rs](#)

Description:

The staking logic intends to enforce that an escrow's `staked_amount` is either 0 or at least `min_staking_amount`.

This invariant is implemented in `validate_escrow_minimum_stake_amount`:

```
pub fn validate_escrow_minimum_stake_amount(&self, escrow: &Escrow) →  
    Result<()> {  
    require!(  
        escrow.staked_amount ≥ self.min_staking_amount  
        || escrow.staked_amount == 0,  
        ReferralError::BelowMinimumStakeAmount  
    );  
    Ok(())  
}
```

The invariant is checked after `stake`, `unstake`, and `immediate_unstake`:

```
pub fn stake(&mut self, escrow: &mut Escrow, amount: u64) → Result<()> {  
    escrow.stake(amount)?;  
    self.config.validate_escrow_minimum_stake_amount(escrow)?;  
    self.staked_amount = self.staked_amount.safe_add(amount)?;  
}
```

```
let actual_unstake_amount = escrow.immediate_unstake(amount)?;  
self.config.validate_escrow_minimum_stake_amount(escrow)?;  
self.staked_amount = self.staked_amount.safe_sub(actual_unstake_amount)?;
```

```
pub fn unstake(&mut self, escrow: &mut Escrow, amount: u64) → Result<u64> {
```

```
let actual_unstake_amount = escrow.unstake(amount)?;
self.config.validate_escrow_minimum_stake_amount(escrow)?;
```

However, `cancel_unstake` restores stake without calling the same validation:

```
pub fn cancel_unstake(&mut self, escrow: &mut Escrow, amount: u64) →
    Result<()> {
    escrow.cancel_unstake(amount)?;

    self.staked_amount = self.staked_amount.safe_add(amount)?;
    self.partial_unstaked_amount
    = self.partial_unstaked_amount.safe_sub(amount)?;
    Ok(())
}
```

As a result, a user can bypass the minimum stake invariant:

1. Minimum stake is 10.
2. User stakes 50.
3. User normally unstakes 5, leaving 45 staked.
4. User immediately unstakes the remaining 45, leaving 0 staked, which is allowed.
5. User cancels the pending unstake of 5.
6. Escrow now has 5 staked, which is non-zero and below the minimum.

Testcase:

```
#[test]
fn test_cancel_unstake_can_restore_below_min_after_full_immediate_unstake()
{
    let min_stake = 10;
    let mut fixture = TestFixture::new(86400, min_stake, 500);
    let user = Keypair::new();
    let stake_amount = 50;
    let pending_unstake_amount = 5;
    let remaining_stake_amount = stake_amount - pending_unstake_amount;
    let (escrow, user_ata) = fixture.fund_and_stake(&user, stake_amount);

    set_clock(&mut fixture.svm, 1000);

    let (unstake_kp, result) = fixture.do_unstake(&escrow, &user,
        pending_unstake_amount);
    result.unwrap();
```

```
let params = ImmediateUnstakeParams {
    max_amount: remaining_stake_amount,
    minimum_out_amount: 0,
    padding0: [0u64; 5],
};

let ix = ix_immediate_unstake(
    &escrow,
    &fixture.staking,
    &fixture.token_vault,
    &fixture.token_mint,
    &user_ata,
    &user.pubkey(),
    params,
);

let tx = Transaction::new_signed_with_payer(
    &[ix],
    Some(&user.pubkey()),
    &[&user],
    fixture.svm.latest_blockhash(),
);
fixture.svm.send_transaction(tx).unwrap();

let escrow_state = read_escrow(&fixture.svm, &escrow);
assert_eq!(escrow_state.staked_amount, 0);
assert_eq!(escrow_state.partial_unstaked_amount,
pending_unstake_amount);

let ix = ix_cancel_unstake(
    &unstake_kp.pubkey(),
    &escrow,
    &fixture.staking,
    &user.pubkey(),
    &user.pubkey(),
);

let tx = Transaction::new_signed_with_payer(
    &[ix],
    Some(&user.pubkey()),
    &[&user],
    fixture.svm.latest_blockhash(),
);
fixture.svm.send_transaction(tx).unwrap();

let escrow_state = read_escrow(&fixture.svm, &escrow);
```

```
assert_eq!(escrow_state.staked_amount, pending_unstake_amount);
assert!(escrow_state.staked_amount > 0);
assert!(escrow_state.staked_amount < min_stake);
assert_eq!(escrow_state.partial_unstaked_amount, 0);

let staking_state = read_staking(&fixture.svm, &fixture.staking);
assert_eq!(staking_state.staked_amount, pending_unstake_amount);
assert_eq!(staking_state.partial_unstaked_amount, 0);
}
```

Recommendations:

we recommend calling `validate_escrow_minimum_stake_amount` inside `Staking::cancel_unstake` after `escrow.cancel_unstake(amount)`:

```
pub fn cancel_unstake(&mut self, escrow: &mut Escrow, amount: u64) →
    Result<()> {
    escrow.cancel_unstake(amount)?;
    self.config.validate_escrow_minimum_stake_amount(escrow)?;

    self.staked_amount = self.staked_amount.safe_add(amount)?;
    self.partial_unstaked_amount
    = self.partial_unstaked_amount.safe_sub(amount)?;
    Ok(())
}
```

This ensures cancelling an unstake cannot restore a non-zero balance below the configured minimum stake.

Meteora: Acknowledged.

4.2 Low Risk

A total of 3 low risk findings were identified.

[L-1] Fee can be distributed across thousands of token accounts

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target

- [ix_claim_penalty_fee.rs](#)

Description:

The `claim_penalty_fee` instruction is permissionless and allows any caller to claim accumulated penalty fees for the configured fee receiver. The destination token account is only required to be owned by `staking.config.fee_receiver`.

```
#[account(
    mut,
    constraint = fee_receiver_token_account.owner
    == staking.load()?.config.fee_receiver @
    ReferralError::InvalidFeeReceiver,
    token::mint = token_mint.key(),
    token::token_program = token_program.key(),
)]
pub fee_receiver_token_account: Box<Account<'info, TokenAccount>>,
```

The instruction then transfers the requested amount of claimed fees to the caller-provided token account:

```
let claimed_fee = staking_state.claim_penalty_fee(params.max_amount)?;

if claimed_fee > 0 {
    transfer_from_pool_to_user(
        &ctx.accounts.staking_authority,
        &ctx.accounts.token_mint,
        &ctx.accounts.token_vault,
```

```
        &ctx.accounts.fee_receiver_token_account,  
        &ctx.accounts.token_program,  
        claimed_fee,  
    )?;  
}
```

Because normal SPL token accounts can be created by anyone while setting another address as the token account owner, an attacker can create many token accounts for the configured fee receiver and repeatedly call `claim_penalty_fee` with small `max_amount` values. This would spread the accrued penalty fees across hundreds or thousands of token accounts controlled by the fee receiver.

The attacker does not gain control of the funds, since the token accounts are still owned by the configured fee receiver. However, this can create operational overhead for the fee receiver, who may need to discover and consolidate many fragmented balances.

Recommendations:

Require `fee_receiver_token_account` to be the canonical associated token account for `staking.config.fee_receiver` and `token_mint`, instead of accepting any token account owned by the fee receiver. This ensures all claimed penalty fees are routed to a single predictable account.

Meteora: Resolved with [PR-13](#).

Zenith: Verified.

[L-2] Users can be locked in after staking

SEVERITY: Low

IMPACT: Medium

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [update_config](#) applies owner-controlled unstaking terms
- [StakingConfig::validate](#) permits up to 9,999 bps and 1 year cooldown
- [unstake](#) computes withdrawal time from current staking config
- [immediate_unstake](#) applies the current staking penalty

Description:

The staking owner can unilaterally change the unstaking terms for already-staked users, potentially trapping them under significantly worse exit conditions than those they accepted when staking.

The protocol allows the staking owner to call `update_config` and change both `unstake_cooldown_seconds` and `immediate_unstake_penalty_bps`:

```
let unstake_cooldown_seconds =  
  new_unstake_cooldown_seconds.unwrap_or(  
    config.unstake_cooldown_seconds);  
  
let immediate_unstake_penalty_bps =  
  new_immediate_unstake_penalty_fee_rate_bps  
    .unwrap_or(config.immediate_unstake_penalty_bps);  
  
StakingConfig::validate(immediate_unstake_penalty_bps,  
  unstake_cooldown_seconds)?;  
  
config.unstake_cooldown_seconds = unstake_cooldown_seconds;  
config.immediate_unstake_penalty_bps = immediate_unstake_penalty_bps;
```

These values are only bounded by `StakingConfig::validate`, which permits an immediate unstake penalty up to 9,999 bps and an unstake cooldown up to 365 days:

```
require!(  
  immediate_unstake_penalty_bps < MAX_FEE_BASIS_POINTS,
```



```
ReferralError::InvalidInputParams
);

require!(
    unstake_cooldown_seconds ≤ MAXIMUM_UNSTAKE_COOLDOWN_SECONDS,
    ReferralError::InvalidInputParams
);
```

When a user performs a normal unstake, the `withdrawable_timestamp` is calculated using the current staking config, not a config snapshot from when the user staked:

```
let current_timestamp = Clock::get()?.unix_timestamp.safe_cast()?;
let withdrawable_timestamp = staking_state
    .config
    .get_withdrawable_timestamp(current_timestamp)?;
```

Similarly, if the user chooses `immediate_unstake`, the currently configured penalty is applied:

```
.config
.charge_immediate_unstake_penalty(actual_unstake_amount)?;
```

As a result, a user may stake into a pool that advertises a 1-day cooldown and a 1% immediate unstake fee. After the user deposits, the staking owner can raise the cooldown to 1 year and the immediate unstake fee to 99.99%. The user is then forced to either wait for the new 1-year cooldown or immediately exit while forfeiting nearly all of their stake as a penalty.

Although `immediate_unstake` includes `minimum_out_amount` slippage protection, this only protects a single immediate-unstake transaction from executing under an unexpectedly high fee. It does not preserve the original staking terms or give the user a way to exit under the terms they deposited into.

Recommendations:

We recommend storing the staking terms accepted by the user in the escrow state and using those escrow-level terms when the user exits.

When an escrow receives its first stake, snapshot the current `unstake_cooldown_seconds` and `immediate_unstake_penalty_bps` into the Escrow account. The protocol should then use the escrow's saved cooldown when computing `withdrawable_timestamp` and the escrow's saved penalty bps when calculating the immediate unstake fee.

Refresh the escrow's saved terms only when the escrow is fully empty or when the user explicitly opts into the new staking terms. This prevents later `update_config` calls from

retroactively worsening exit conditions for already-staked users while still allowing updated terms for new deposits.

Meteora: Acknowledged.

[L-3] Protocol-sponsored escrow rent can be captured by owner on close

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Medium

Target

- [programs/referral/src/instructions/permissionless/ix_close_escrow.rs#L26](#)

Description:

If a third party (for example, protocol infrastructure) pays rent to create escrow accounts, the escrow owner can later close the escrow and route rent refunds to an arbitrary address. The protocol cannot enforce rent repayment at the program level.

```
// programs/referral/src/instructions/permissionless/ix_initialize_escrow.rs
#[account(
    init,
    seeds = [seeds::ESCROW, staking.key().as_ref(), owner.key().as_ref()],
    bump,
    payer = payer,
    space = Escrow::SPACE,
)]
pub escrow: AccountLoader<'info, Escrow>,
```

```
// programs/referral/src/instructions/permissionless/ix_close_escrow.rs
pub owner: Signer<'info>,

/// CHECK: Account to receive the rent lamports from closing the escrow
account.
#[account(mut)]
|> pub rent_receiver: UncheckedAccount<'info>,

...
ctx.accounts
    .escrow
|> .close(ctx.accounts.rent_receiver.to_account_info())?;
```

Impacts: Protocol or relayer rent subsidies can be systematically extracted by escrow owners at account close. Per-account loss is bounded to escrow rent, but cumulative cost can grow with large-scale sponsored onboarding.

Recommendations:

(1) If protocol-sponsored rent is expected, add a `rent_payer` field to `Escrow` and set it at initialization. (2) Restrict close-time refund routing by enforcing `rent_receiver = escrow.rent_payer` for strict refund semantics. (3) If flexible routing is desired, make it explicit in docs and product policy, and avoid protocol-funded rent without off-chain reimbursement controls. For example,

```
// programs/referral/src/states/escrow.rs
#[account(zero_copy)]
pub struct Escrow {
    pub staking: Pubkey,
    pub owner: Pubkey,
    pub staked_amount: u64,
    // @audit New field to bind rent refund destination.
    pub rent_payer: Pubkey,
    ...
}

impl Escrow {
    pub fn initialize(&mut self, staking: Pubkey, owner: Pubkey, rent_payer: Pubkey) {
        self.staking = staking;
        self.owner = owner;
        self.rent_payer = rent_payer;
        ...
    }
}
```

```
// programs/referral/src/instructions/permissionless/ix_close_escrow.rs
pub fn handle_close_escrow(
    ctx: Context<CloseEscrowCtx>,
    skip_close_if_non_empty: bool,
) -> Result<()> {
    let escrow_state = ctx.accounts.escrow.load()?;

    // @audit Enforce refund goes back to original rent sponsor.
    require_keys_eq!(
        ctx.accounts.rent_receiver.key(),
        escrow_state.rent_payer,
        ReferralError::InvalidRentReceiver
    )
}
```

```
);  
  
...  
drop(escrow_state);  
ctx.accounts  
  .escrow  
  .close(ctx.accounts.rent_receiver.to_account_info());  
Ok()  
}
```

Meteora: Acknowledged.

4.3 Informational

A total of 10 informational findings were identified.

[\[I-1\] Division and remainder return MathOverflow on division-by-zero](#)

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target

- [programs/referral/src/math/safe_math.rs#L62](#)
- [programs/referral/src/math/safe_math.rs#L74](#)

Description:

safe_div and safe_rem map checked_div and checked_rem failures to ReferralError::MathOverflow. For u64 and u128, these failures are division-by-zero cases, not arithmetic overflow.

```
// programs/referral/src/math/safe_math.rs
fn safe_div(self, v: $t) → Result<$t, ReferralError> {
    match self.checked_div(v) {
        Some(result) ⇒ Ok(result),
    |>    None ⇒ Err(ReferralError::MathOverflow),
    }
}

fn safe_rem(self, v: $t) → Result<$t, ReferralError> {
    match self.checked_rem(v) {
        Some(result) ⇒ Ok(result),
    |>    None ⇒ Err(ReferralError::MathOverflow),
    }
}
```

Recommendations:

1. Add a dedicated error variant for division-by-zero in `ReferralError`.
2. Return that variant from `safe_div` and `safe_rem` when checked operations return `None`.

Meteora: Acknowledged.

[I-2] Unused Error ReferralError::UnsupportedMintExtension

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [errors.rs](#)

Description:

ReferralError::UnsupportedMintExtension appears to be leftover dead code. The program does not support Token-2022 mint extensions, and no instruction or validation path ever returns this error. Instead, token mint validation only checks whether the mint is owned by the expected token program and returns UnsupportedTokenMint on failure.

```
#[error_code]
pub enum ReferralError {
    #[msg("Unsupported token mint")]
    UnsupportedTokenMint,

    #[msg("Unsupported mint extension")]
    UnsupportedMintExtension,
}
```

The only observed token compatibility check rejects unsupported token mints directly:

```
require!(
    token_mint.to_account_info().owner == token_program.key,
    ReferralError::UnsupportedTokenMint
);
```

Because Token-2022 extensions are not supported or inspected, the UnsupportedMintExtension variant is unreachable and may confuse maintainers or integrators about the program's actual token support model.

Recommendations:

Remove `ReferralError::UnsupportedMintExtension` to keep the error surface aligned with the program's implemented Token Program support.

Meteora: Resolved with [PR-12](#).

Zenith: Verified.

[I-3] Unstakes can be DOSd

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [ix_stake.rs](#)
- [ix_unstake.rs](#)
- [ix_immediate_unstake.rs](#)
- [staking.rs](#)
- [escrow.rs](#)

Description:

The protocol enforces that after an unstake, an escrow's remaining staked amount must either be zero or at least `min_staking_amount`.

```
pub fn validate_escrow_minimum_stake_amount(&self, escrow: &Escrow) →  
    Result<()> {  
    require!(  
        escrow.staked_amount ≥ self.min_staking_amount  
        || escrow.staked_amount == 0,  
        ReferralError::BelowMinimumStakeAmount  
    );  
    Ok(())  
}
```

This check is applied after both delayed unstaking and immediate unstaking.

```
pub fn unstake(&mut self, escrow: &mut Escrow, amount: u64) → Result<u64> {  
    let actual_unstake_amount = escrow.unstake(amount)?;  
    self.config.validate_escrow_minimum_stake_amount(escrow)?;  
  
    self.staked_amount  
    = self.staked_amount.safe_sub(actual_unstake_amount)?;  
    self.partial_unstaked_amount = self  
        .partial_unstaked_amount  
        .safe_add(actual_unstake_amount)?;
```

```
Ok(actual_unstake_amount)
}
```

```
pub fn immediate_unstake(
    &mut self,
    escrow: &mut Escrow,
    amount: u64,
) → Result<ImmediateUnstakeResult> {
    let actual_unstake_amount = escrow.immediate_unstake(amount)?;
    self.config.validate_escrow_minimum_stake_amount(escrow)?;
    self.staked_amount
    = self.staked_amount.safe_sub(actual_unstake_amount)?;

    // ...
}
```

The unstake amount is capped by the escrow's current staked_amount.

```
pub fn unstake(&mut self, max_amount: u64) → Result<u64> {
    let capped_amount = std::cmp::min(max_amount, self.staked_amount);
    self.staked_amount = self.staked_amount.safe_sub(capped_amount)?;
    self.partial_unstaked_amount
    = self.partial_unstaked_amount.safe_add(capped_amount)?;
    Ok(capped_amount)
}
```

However, the staking instruction intentionally allows anyone to stake into an existing escrow.

```
#[account(
    mut,
    has_one = staking,
    // Note: We allow anyone to stake into the escrow, so we don't check the
    owner here
)]
pub escrow: AccountLoader<'info, Escrow>,
```

As a result, a third party can stake a very small amount into a user's escrow before the user's unstake transaction executes. If the user attempts to fully unstake by passing their exact observed balance as max_amount, the attacker's top-up increases the escrow balance. The user's transaction then unstakes only the exact max_amount, leaving the attacker's small remainder in the escrow.

If that remainder is below min_staking_amount, the transaction reverts with BelowMinimumStakeAmount. This creates a low-cost denial of service against exact-balance

full unstake flows.

Users can avoid this by passing `u64::MAX`, which causes the protocol to unstake the full escrow balance at execution time. However, this behavior may not be obvious to integrators or users unless it is clearly documented and reflected in the interface.

Recommendations:

Clearly document that full unstake operations must pass `u64::MAX` rather than a cached exact escrow balance. This should be included in developer documentation, SDKs, frontend code, and integration examples.

User-facing interfaces should make this behavior explicit. Any “unstake all” or “full exit” button should automatically pass `u64::MAX`, not the currently displayed escrow balance, to prevent denial of service through small third-party stake top-ups.

Consider exposing a dedicated `unstake_all` / `immediate_unstake_all` helper in the SDK or interface layer so integrators do not need to know about the `u64::MAX` sentinel value.

Meteora: Resolved with [PR-11](#).

Zenith: Verified.

[I-4] Missing minimum cooldown bound: intent-enforcement mismatch and protocol policy inconsistency

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/referral/src/states/staking.rs#L179](#)

Description:

StakingConfig::validate enforces an upper bound on unstake_cooldown_seconds via MAXIMUM_UNSTAKE_COOLDOWN_SECONDS (1 year) but no meaningful lower bound — `> 0` is the only guard, meaning 1 second is a valid configuration, which doesn't meaningfully enforce a cool-down period.

```
// programs/referral/src/states/staking.rs
} else {
    // When there's a penalty for immediate unstake, we require cooldown
    // period to be greater than 0
    // to make the penalty meaningful. Otherwise users will just do partial
    // unstake and immediately
    // withdraw to get around the penalty.
    require!(
        unstake_cooldown_seconds > 0, // 1 second satisfies this; stated
        goal is not achieved
        ReferralError::InvalidInputParams
    );
}
```

With `cooldown = 1s` and any `penalty_bps > 0`, any staker avoids the penalty at negligible cost with next to zero commitment.

Impacts:

- Protocol policy inconsistency: Meteora's Stake2Earn documents a 6-hour minimum cooldown as the protocol-level floor for meaningful staking commitment. This program has no comparable enforcement.
- Off-chain reward risk: If any off-chain reward logic (snapshots, yield distribution, eligibility scoring) trusts this contract's staked balances as commitment evidence, a

1-second cooldown pool enables flash-stake like behavior, which might not be accounted for in downstream rewarding logic systems.

Recommendations:

Define `MINIMUM_UNSTAKE_COOLDOWN_SECONDS` in `constants.rs` and ensure a minimal bound for `unstake_cooldown_seconds`. For example,

```
// programs/referral/src/constants.rs
pub const MAXIMUM_UNSTAKE_COOLDOWN_SECONDS: u64 = 365 * 24 * 3600; // 1 year
pub const MINIMUM_UNSTAKE_COOLDOWN_SECONDS: u64 = 21_600;          // 6
                                hours (protocol floor)
```

```
// programs/referral/src/states/staking.rs
} else {
    // Cooldown must meet the protocol minimum to ensure the penalty is
    // meaningful.
    require!(
        unstake_cooldown_seconds ≥ MINIMUM_UNSTAKE_COOLDOWN_SECONDS,
        ReferralError::InvalidInputParams
    );
}
```

Meteora: Resolved with [@89fb8b7a3e ...](#)

Zenith: Verified.

[I-5] Flash loan enables event inflation in frictionless staking pools

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [programs/referral/src/states/staking.rs#L169-L172](#)

Description:

When a staking pool is configured with `immediate_unstake_penalty_bps = 0` and `unstake_cooldown_seconds = 0`, `stake` and `immediate_unstake` can be called atomically in the same transaction with no economic cost beyond the flash-loan fee (~0.001%). On-chain balances net to zero after the round trip, but `StakeEvt` and `ImmediateUnstakeEvt` are permanently emitted with flash-loan-borrowed notionals.

```
// programs/referral/src/instructions/permissionless/ix_immediate_unstake.rs
pub fn handle_immediate_unstake(...) -> Result<()> {
    params.validate()?;
    // @audit no check on when the stake was placed
    require!(escrow_state.staked_amount > 0,
        ReferralError::NothingToUnstake);
    let ImmediateUnstakeResult { user_receive_amount, penalty_fee_amount } =
        staking_state.immediate_unstake(&mut escrow_state, max_amount)?;
    emit_cpi!(ImmediateUnstakeEvt {
        staking_amount_latest, // @audit inflated transient notional
        recorded permanently
        penalty_fee_amount,
        ...
    });
}
```

Impacts: Impact is conditional on off-chain consumer design. If downstream reward/indexing logic naively attributes stake weight from gross event notional without same-transaction netting or on-chain state reconciliation, an attacker can inflate their apparent stake repeatedly at negligible cost.

Recommendations:

1. Consider enforcing a non-atomic boundary between `stake` and `immediate_unstake`, while preserving `immediate_unstake` as an early-exit path for existing stake. For example, track the most recent stake slot on Escrow and require `immediate_unstake` to execute only in a later slot (`current_slot > last_staked_slot`). This prevents same-transaction flash-loan round-trips from emitting large transient stake notionals.
2. Or document the corresponding integration requirement for downstream consumers: staking/reward accounting must treat events as state transitions, not standalone proof of sustained stake. Consumers should reconcile stake deltas per transaction/signature and ensure same-transaction `stake → immediate_unstake` round-trips contribute zero stake-duration or reward weight.

Meteora: Acknowledged.

[I-6] Missing Pubkey::default() guard on fee receiver and owner configuration paths

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/referral/src/instructions/permissionless/ix_initialize_staking.rs#L83](#)
- [programs/referral/src/instructions/permissionless/ix_initialize_staking.rs#L80](#)
- [programs/referral/src/instructions/permissioned/ix_update_penalty_fee_receiver.rs#L22](#)

Description:

Two persistent pubkeys (staking_penalty_fee_receiver and owner) are stored without any guard rejecting Pubkey::default() or other trivially uncontrollable keys. Both fields gate critical capabilities (fee distribution and admin control), so writing an unrecoverable key to either permanently impairs the corresponding functionality for that staking instance.

Recommendations:

1. Add a Pubkey::default() rejection for owner and staking_penalty_fee_receiver in handle_initialize_staking before writing state.
2. Add a Pubkey::default() rejection for new_fee_receiver in handle_update_penalty_fee_receiver.
3. Add a Pubkey::default() rejection for new_owner in transfer_owner constraints (or handler).

For example,

```
// programs/referral/src/instructions/permission-  
less/ix_initialize_staking.rs  
pub fn handle_initialize_staking(  
    ctx: Context<InitializeStakingCtx>,  
    params: InitializeStakingParams,  
) -> Result<()> {  
    ...  
    require!(owner != Pubkey::default(), ReferralError::InvalidNewOwner);
```

```
require!(
    staking_penalty_fee_receiver ≠ Pubkey::default(),
    ReferralError::InvalidFeeReceiver
);
```

```
// programs/referral/src/instructions/permissioned/ix_update_penalty_fee_receiver.rs
pub fn handle_update_penalty_fee_receiver(ctx:
    Context<UpdatePenaltyFeeReceiverCtx>) → Result<()> {
    let mut staking_state = ctx.accounts.staking.load_mut()?;
    let old_fee_receiver = staking_state.config.fee_receiver;
    let new_fee_receiver = ctx.accounts.new_fee_receiver.key();

    require!(new_fee_receiver ≠ old_fee_receiver,
        ReferralError::InvalidFeeReceiver);
    require!(new_fee_receiver ≠ Pubkey::default(),
        ReferralError::InvalidFeeReceiver);

    staking_state.config.fee_receiver = new_fee_receiver;
    Ok(())
}
```

```
// programs/referral/src/instructions/permissioned/ix_transfer_owner.rs
pub struct TransferOwnerCtx<'info> {
    ...
    #[account(
        constraint = new_owner.key.ne(owner.key) @
        ReferralError::InvalidNewOwner,
        constraint = new_owner.key() ≠ Pubkey::default() @
        ReferralError::InvalidNewOwner,
    )]
    pub new_owner: UncheckedAccount<'info>,
}
```

Meteora: Resolved with [@a8a04ead78...](#)

Zenith: Verified.

[I-7] Manual AccountLoader::close path misses close self-target check

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/referral/src/instructions/permissionless/ix_close_escrow.rs#L51](#)

Description:

`ix_close_escrow` manually calls `AccountLoader::close` instead of using Anchor `#[account(close = ...)]`. This manual path does not include Anchor's generated `ConstraintClose` guard that rejects `account_to_close = rent_receiver`.

```
// programs/referral/src/instructions/permissionless/ix_close_escrow.rs
pub fn handle_close_escrow(
    ctx: Context<CloseEscrowCtx>,
    skip_close_if_non_empty: bool,
) -> Result<()> {
    let escrow_state = ctx.accounts.escrow.load()?;
    let mut staking_state = ctx.accounts.staking.load_mut()?;

    if !escrow_state.is_empty() {
        require!(
            skip_close_if_non_empty,
            ReferralError::CannotCloseNonEmptyEscrow
        );
        return Ok(());
    }

    staking_state.metrics.decrement_total_escrow();
    drop(escrow_state);
    ctx.accounts
        .escrow
        .close(ctx.accounts.rent_receiver.to_account_info()?);
    // ...
    Ok(())
}
```

In comparison, the anchor' close path checks `account_to_close` and `rent_receiver`.

```
pub fn generate_constraint_close(
  f: &Field,
  c: &ConstraintClose,
  accs: &AccountsStruct,
) → proc_macro2::TokenStream {
  // ...
  quote! {
    {
      #target_optional_check
    }
  }
  |> if #field.key() == #target.key() {
    return Err(anchor_lang::error::Error::from(
      anchor_lang::error::ErrorCode::ConstraintClose)
      .with_account_name(#name_str));
  }
  ...
}
```

Impacts:

- If `rent_receiver` is set to the escrow account itself, the close path lacks the expected Anchor safety check.
- This can produce unintended close behavior (self-target close edge case) and inconsistent rent-return semantics versus other instructions that use `close = ...`.

Recommendations:

Add an explicit key-inequality guard before manual close in `handle_close_escrow`:
`require_keys_neq!(ctx.accounts.escrow.key(), ctx.accounts.rent_receiver.key(),
ReferralError::InvalidInputParams);`(Or custom error)

Meteora: Resolved with [@3165c0dab1 ...](#)

Zenith: Verified.

[I-8] Single-step ownership transfer without new-owner confirmation

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target

- [programs/referral/src/instructions/permissioned/ix_transfer_owner.rs#L17](#)

Description:

`transfer_owner` immediately overwrites `staking.owner` with the caller-supplied `new_owner` key in a single transaction. The only on-chain guard is that `new_owner` must differ from the current owner. There is no pending-owner stage and no acceptance step that would require the intended new owner to sign before the transfer takes effect. A single mistaken or social-engineered call by the current owner could replace ownership with no on-chain recovery path.

```
// programs/referral/src/instructions/permissioned/ix_transfer_owner.rs
#[account(
    constraint = new_owner.key.ne(owner.key) @ ReferralError::InvalidNewOwner
)]
/// CHECK: New owner can be any account, no need to check
pub new_owner: UncheckedAccount<'info>,
```

```
// programs/referral/src/instructions/permissioned/ix_transfer_owner.rs
pub fn handle_transfer_owner(ctx: Context<TransferOwnerCtx>) → Result<()> {
    let staking_state = &mut ctx.accounts.staking.load_mut()?;
    staking_state.owner = ctx.accounts.new_owner.key();
    ...
}
```

Recommendations:

Introduce a two-step transfer: `transfer_owner` stores the proposed key in a `pending_owner` field on `Staking` without changing owner; a new `accept_owner` instruction, callable only by the signer matching `pending_owner`. For example,

```
// programs/referral/src/states/staking.rs
pub struct Staking {
    pub owner: Pubkey,
    // @audit add pending slot; cleared after accept or on cancellation
    pub pending_owner: Pubkey,
    ...
}
```

```
// programs/referral/src/instructions/permissioned/ix_transfer_owner.rs
pub fn handle_transfer_owner(ctx: Context<TransferOwnerCtx>) → Result<()> {
    let staking_state = &mut ctx.accounts.staking.load_mut()?;
    // @audit write to pending_owner
    staking_state.pending_owner = ctx.accounts.new_owner.key();
    ...
    Ok(())
}
```

```
// programs/referral/src/instructions/permissioned/ix_accept_owner.rs (new)
...
pub fn handle_accept_owner(ctx: Context<AcceptOwnerCtx>) → Result<()> {
    let staking_state = &mut ctx.accounts.staking.load_mut()?;
    let old_owner = staking_state.owner;
    staking_state.owner = staking_state.pending_owner;
    staking_state.pending_owner = Pubkey::default();
    ...
    Ok(())
}
```

Meteora: Acknowledged.

[I-9] Slippage on immediate unstake can malfunction

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [programs/referral/src/instructions/permissionless/ix_immediate_unstake.rs](#)
- [programs/referral/src/instructions/permissionless/ix_stake.rs](#)

Description:

`immediate_unstake` uses an absolute `minimum_out_amount` as slippage protection against changes to the immediate unstake penalty. This does not fully protect the user's intended maximum fee tolerance when the actual unstaked amount can change before execution.

```
pub struct ImmediateUnstakeParams {  
    /// The maximum amount of tokens to unstake. User can pass in u64::MAX  
    /// to unstake the full amount.  
    pub max_amount: u64,  
    /// Slippage protection against penalty fee bps changes. Eg: Update  
    /// penalty fee bps sandwich.  
    pub minimum_out_amount: u64,  
    pub padding0: [u64; 5],  
}
```

The handler computes the actual unstake amount from the escrow's current `staked_amount`, then only checks that the post-fee output is at least `minimum_out_amount`.

```
let ImmediateUnstakeResult {  
    user_receive_amount,  
    penalty_fee_amount,  
} = staking_state.immediate_unstake(&mut escrow_state, max_amount)?;  
  
require!(  
    user_receive_amount ≥ minimum_out_amount,  
    ReferralError::SlippageExceeded  
);
```

However, anyone can stake into an existing escrow because the stake instruction intentionally does not require the escrow owner to sign.

```
#[account(  
  mut,  
  has_one = staking,  
  // Note: We allow anyone to stake into the escrow, so we don't check the  
  owner here  
)]  
pub escrow: AccountLoader<'info, Escrow>,
```

This creates the following edge case:

1. A user has 100 tokens staked and submits an immediate unstake with `max_amount = u64::MAX`.
2. The user accepts at most a 10% penalty and therefore sets `minimum_out_amount = 90`.
3. Before the transaction lands, another party stakes 100 additional tokens into the user's escrow.
4. The immediate unstake penalty is increased to 50%.
5. The user's transaction executes against 200 staked tokens, receives 100 tokens after the 50% penalty, and passes the `minimum_out_amount ≥ 90` check.

The user's intended fee tolerance was 10%, but the transaction succeeds with a 50% fee because the additional stake increased the absolute post-fee output enough to satisfy the minimum amount check.

Recommendations:

Replace or supplement `minimum_out_amount` with a user-supplied `max_immediate_unstake_penalty_bps`.

The instruction should validate the current configured penalty before applying the unstake:

```
pub struct ImmediateUnstakeParams {  
  pub max_amount: u64,  
  pub max_immediate_unstake_penalty_bps: u16,  
  pub padding0: [u64; 5],  
}
```

Then enforce:


```
require!(  
    staking_state.config.immediate_unstake_penalty_bps  
    ≤ params.max_immediate_unstake_penalty_bps,  
    ReferralError::SlippageExceeded  
);
```

This protects the user's actual fee tolerance independently of any changes to the escrow balance between transaction submission and execution.

Meteora: Resolved with [PR-15](#).

Zenith: Verified.

[I-10] Incorrect receiver account emission

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target

- [ix_immediate_unstake.rs](#)
- [ix_withdraw.rs](#)
- [events.rs](#)

Description:

The ImmediateUnstakeEvt and WithdrawEvt events document their receiver field as the token account that receives the unstaked tokens:

```
pub struct ImmediateUnstakeEvt {  
    // ...  
    /// The account that receives the unstake token  
    pub receiver: Pubkey,  
    // ...  
}  
  
pub struct WithdrawEvt {  
    // ...  
    /// The account that receives the unstake token  
    pub receiver: Pubkey,  
    // ...  
}
```

However, both immediate_unstake and withdraw emit the owner of the receiving token account instead of the receiving token account address itself.

In ix_immediate_unstake.rs, tokens are transferred to owner_token_vault, but the emitted receiver is owner_token_vault.owner:

```
transfer_from_pool_to_user(  
    &ctx.accounts.staking_authority,  
    &ctx.accounts.token_mint,  
    &ctx.accounts.token_vault,
```

```
        &ctx.accounts.owner_token_vault,
        &ctx.accounts.token_program,
        user_receive_amount,
    )?;

    let escrow = ctx.accounts.escrow.key();
    let staking = ctx.accounts.staking.key();
    let owner = ctx.accounts.owner.key();
    let receiver = ctx.accounts.owner_token_vault.owner;

    emit_cpi!(ImmediateUnstakeEvt {
        escrow,
        staking,
        owner,
        // ...
        receiver,
        params,
        token_mint,
        actual_unstake_amount,
    });
};
```

The same issue exists in `ix_withdraw.rs`:

```
transfer_from_pool_to_user(
    &ctx.accounts.staking_authority,
    &ctx.accounts.token_mint,
    &ctx.accounts.token_vault,
    &ctx.accounts.owner_token_vault,
    &ctx.accounts.token_program,
    unstake_amount,
)?;

let owner = ctx.accounts.owner.key();
let escrow = ctx.accounts.escrow.key();
let staking = ctx.accounts.staking.key();
let unstake = ctx.accounts.unstake.key();
let receiver = ctx.accounts.owner_token_vault.owner;

emit_cpi!(WithdrawEvt {
    owner,
    escrow,
    staking,
    unstake,
    unstake_amount,
    rent_receiver,
    receiver,
});
```

```
        escrow_amount_latest,  
        staking_amount_latest,  
        token_mint,  
    });
```

As a result, event consumers cannot reliably reconcile the emitted `receiver` field with the actual token account that received funds. This is especially misleading when users receive tokens into arbitrary associated or non-associated token accounts, because the event logs the token account owner rather than the concrete account whose balance changed.

Recommendations:

Emit the receiving token account address in both handlers:

```
let receiver = ctx.accounts.owner_token_vault.key();
```

Apply this change in both `ix_immediate_unstake.rs` and `ix_withdraw.rs`. Also update or add tests that assert `ImmediateUnstakeEvt.receiver` and `WithdrawEvt.receiver` equal the destination token account public key, not the token account owner.

Meteora: Resolved with [PR-14](#).

Zenith: Verified.